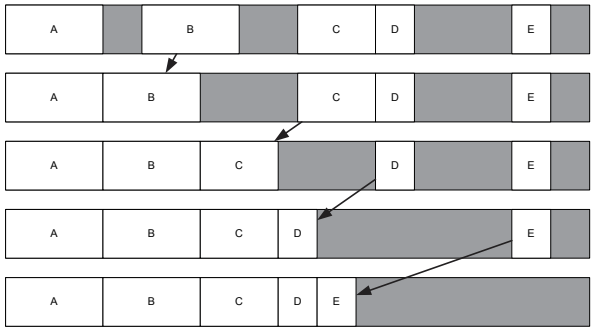


Figure 6.7. Defragmentation by shifting allocated blocks to lower addresses.

The solution to this problem is to patch any and all pointers into a shifted memory block so that they point to the correct new address after the shift. This procedure is known as pointer *relocation*. Unfortunately, there is no general-purpose way to *find* all the pointers that point into a particular region of memory. So if we are going to support memory defragmentation in our game engine, programmers must either carefully keep track of all the pointers manually so they can be relocated, or pointers must be abandoned in favor of something inherently more amenable to relocation, such as *smart pointers* or *handles*.

A smart pointer is a small class that contains a pointer and acts like a pointer for most intents and purposes. But because a smart pointer is a class, it can be coded to handle memory relocation properly. One approach is to arrange for all smart pointers to add themselves to a global linked list. Whenever a block of memory is shifted within the heap, the linked list of all smart pointers can be scanned, and each pointer that points into the shifted block of memory can be adjusted appropriately.

A handle is usually implemented as an index into a non-relocatable table, which itself contains the pointers. When an allocated block is shifted in memory, the handle table can be scanned and all relevant pointers found and updated automatically. Because the handles are just indices into the pointer table, their values never change no matter how the memory blocks are shifted, so the objects that use the handles are never affected by memory relocation.

Another problem with relocation arises when certain memory blocks cannot be relocated. For example, if you are using a third-party library that does not use smart pointers or handles, it's possible that any pointers into its data structures will not be relocatable. The best way around this problem is usually to arrange for the library in question to allocate its memory from a special buffer outside of the relocatable memory area. The other option is to simply

accept that some blocks will not be relocatable. If the number and size of the non-relocatable blocks are both small, a relocation system will still perform quite well.

It is interesting to note that all of Naughty Dog's engines have supported defragmentation. Handles are used wherever possible to avoid the need to relocate pointers. However, in some cases raw pointers cannot be avoided. These pointers are carefully tracked and relocated manually whenever a memory block is shifted due to defragmentation. A few of Naughty Dog's game object classes are not relocatable for various reasons. However, as mentioned above, this doesn't pose any practical problems, because the number of such objects is always very small, and their sizes are tiny when compared to the overall size of the relocatable memory area.

#### *Amortizing Defragmentation Costs*

Defragmentation can be a slow operation because it involves copying memory blocks. However, we needn't fully defragment the heap all at once. Instead, the cost can be amortized over many frames. We can allow up to $N$ allocated blocks to be shifted each frame, for some small value of $N$ like 8 or 16. If our game is running at 30 frames per second, then each frame lasts $1/30$ of a second (33 ms). So, the heap can usually be completely defragmented in less than one second without having any noticeable effect on the game's frame rate. As long as allocations and deallocations aren't happening at a faster rate than the defragmentation shifts, the heap will remain mostly defragmented at all times.

This approach is only valid when the size of each block is relatively small, so that the time required to move a single block does not exceed the time allotted to relocation each frame. If very large blocks need to be relocated, we can often break them up into two or more subblocks, each of which can be relocated independently. This hasn't proved to be a problem in Naughty Dog's engine, because relocation is only used for dynamic game objects, and they are never larger than a few kibibytes—and usually much smaller.

## 6.3 Containers

Game programmers employ a wide variety of collection-oriented data structures, also known as *containers* or *collections*. The job of a container is always the same—to house and manage zero or more data elements; however, the details of how they do this vary greatly, and each type of container has its pros and cons. Common container data types include, but are certainly not limited

to, the following.

- *Array*. An ordered, contiguous collection of elements accessed by index. The length of the array is usually statically defined at compile time. It may be multidimensional. C and C++ support these natively (e.g., `int a[5]`).
- *Dynamic array*. An array whose length can change dynamically at runtime (e.g., the C++ standard library's `std::vector`).
- *Linked list*. An ordered collection of elements not stored contiguously in memory but rather linked to one another via pointers (e.g., the C++ standard library's `std::list`).
- *Stack*. A container that supports the last-in-first-out (LIFO) model for adding and removing elements, also known as push/pop (e.g., `std::stack`).
- *Queue*. A container that supports the first-in-first-out (FIFO) model for adding and removing elements (e.g., `std::queue`).
- *Deque*. A double-ended queue—supports efficient insertion and removal at both ends of the array (e.g., `std::deque`).
- *Tree*. A container in which elements are grouped hierarchically. Each element (node) has zero or one parent and zero or more children. A tree is a special case of a DAG (see below).
- *Binary search tree (BST)*. A tree in which each node has at most two children, with an order property to keep the nodes sorted by some well-defined criteria. There are various kinds of binary search trees, including red-black trees, splay trees, AVL trees, etc.
- *Binary heap*. A binary tree that maintains itself in sorted order, much like a binary search tree, via two rules: the *shape property*, which specifies that the tree must be fully filled and that the last row of the tree is filled from left to right; and the *heap property*, which states that every node is, by some user-defined criterion, "greater than" or "equal to" all of its children.
- *Priority queue*. A container that permits elements to be added in any order and then removed in an order defined by some property of the elements themselves (i.e., their *priority*). A priority queue is typically implemented as a *heap* (e.g., `std::priority_queue`), but other implementations are possible. A priority queue is a bit like a list that stays sorted at all times, except that a priority queue only supports retrieval of the highest-priority element, and it is rarely implemented as a list under the hood.

- *Dictionary*. A table of key-value pairs. A value can be "looked up" efficiently given the corresponding key. A dictionary is also known as a *map* or *hash table*, although technically a hash table is just one possible implementation of a dictionary (e.g., `std::map`, `std::hash_map`).
- *Set*. A container that guarantees that all elements are unique according to some criteria. A set acts like a dictionary with only keys, but no values.
- *Graph*. A collection of nodes connected to one another by unidirectional or bidirectional pathways in an arbitrary pattern.
- *Directed acyclic graph (DAG)*. A collection of nodes with unidirectional (i.e., *directed*) interconnections, with no *cycles* (i.e., there is no nonempty path that starts and ends on the same node).

### 6.3.1 Container Operations

Game engines that make use of container classes inevitably make use of various commonplace algorithms as well. Some examples include:

- *Insert*. Add a new element to the container. The new element might be placed at the beginning of the list, or the end, or in some other location; or the container might not have a notion of ordering at all.
- *Remove*. Remove an element from the container; this may require a find operation (see below). However, if an iterator is available that refers to the desired element, it may be more efficient to remove the element using the iterator.
- *Sequential access (iteration)*. Accessing each element of the container in some "natural" predefined order.
- *Random access*. Accessing elements in the container in an arbitrary order.
- *Find*. Search a container for an element that meets a given criterion. There are all sorts of variants on the find operation, including finding in reverse, finding multiple elements, etc. In addition, different types of data structures and different situations call for different algorithms (see http://en.wikipedia.org/wiki/Search_algorithm).
- *Sort*. Sort the contents of a container according to some given criteria. There are many different sorting algorithms, including bubble sort, selection sort, insertion sort, quicksort and so on. (See http://en.wikipedia.org/wiki/Sorting_algorithm for details.)

### 6.3.2 Iterators

An iterator is a little class that "knows" how to efficiently visit the elements in a particular kind of container. It acts like an array index or pointer—it refers to one element in the container at a time, it can be advanced to the next element, and it provides some sort of mechanism for testing whether or not all elements in the container have been visited. As an example, the first of the following two code snippets iterates over a C-style array using a pointer, while the second iterates over a linked list using almost identical syntax.

```
void processArray(int container[], int numElements)
{
    int* pBegin = &container[0];
    int* pEnd = &container[numElements];

    for (int* p = pBegin; p != pEnd; p++)
    {
        int element = *p;
        // process element...
    }
}

void processList(std::list<int>& container)
{
    std::list<int>::iterator pBegin = container.begin();
    std::list<int>::iterator pEnd = container.end();

    for (auto p = pBegin; p != pEnd; ++p)
    {
        int element = *p;
        // process element...
    }
}
```

The key benefits to using an iterator over attempting to access the container's elements directly are as follows:

- Direct access would break the container class' encapsulation. An iterator, on the other hand, is typically a *friend* of the container class, and as such it can iterate efficiently without exposing any implementation details to the outside world. (In fact, most good container classes hide their internal details and cannot be iterated over *without* an iterator.)
- An iterator can simplify the process of iterating. Most iterators act like array indices or pointers, so a simple loop can be written in which the

iterator is incremented and compared against a terminating condition—even when the underlying data structure is arbitrarily complex. For example, an iterator can make an in-order depth-first tree traversal look no more complex than a simple array iteration.

#### 6.3.2.1 Preincrement versus Postincrement

Notice in the `processArray()` example that we are using C++'s *postincrement* operator, `p++`, rather than the *preincrement* operator, `++p`. This is a subtle but sometimes important optimization. The preincrement operator increments the contents of the variable *before* its (now modified) value is used in the expression. The postincrement operator increments the contents of the variable *after* it has been used. This means that writing `++p` introduces a *data dependency* into your code—the CPU must wait for the increment operation to be completed before its value can be used in the expression. On a deeply pipelined CPU, this introduces a *stall*. On the other hand, with `p++` there is no data dependency. The value of the variable can be used immediately, and the increment operation can happen later or in parallel with its use. Either way, no stall is introduced into the pipeline.

Of course, within the "update" expression of a `for` loop, there should be no difference between pre- and postincrement. This is because any good compiler will recognize that the *value* of the variable isn't used in `update_expr`. But in cases where the value *is* used, postincrement is preferable because it doesn't introduce a stall in the CPU's pipeline.

It can be wise to make an exception to this little rule of thumb for classes with *overloaded* increment operators, as is common practice in *iterator* classes. By definition, the postincrement operator must return an unmodified *copy* of the object on which it is called. Depending on the size and complexity of the data members of the class, the added cost of copying the iterator can tip the scales toward a preference for preincrement when using such classes in performance-critical loops. (Preincrement isn't necessarily better than postincrement in such a simple example as the function `processList()` shown above, but I've implemented it with preincrement to highlight the difference.)

### 6.3.3 Algorithmic Complexity

The choice of which container type to use for a given application depends upon the performance and memory characteristics of the container being considered. For each container type, we can determine the theoretical performance of common operations such as insertion, removal, find and sort.

We usually indicate the amount of time $T$ that an operation is expected to

take as a function of the number of elements $n$ in the container:

$$T = f(n).$$

Rather than try to find the exact function $f$, we concern ourselves only with finding the overall *order* of the function. For example, if the actual theoretical function were any of the following,

$$T = 5n^2 + 17,$$

$$T = 102n^2 + 50n + 12,$$
$$T = \tfrac{1}{2}n^2,$$

we would, in all cases, simplify the expression down to its most relevant term—in this case $n^2$. To indicate that we are only stating the *order* of the function, not its exact equation, we use "big O" notation and write

$$T = O(n^2).$$

The order of an algorithm can usually be determined via an inspection of the pseudocode. If the algorithm's execution time is not dependent upon the number of elements in the container at all, we say it is $O(1)$ (i.e., it completes in *constant time*). If the algorithm performs a *loop* over the elements in the container and visits each element once, such as in a linear search of an unsorted list, we say the algorithm is $O(n)$. If two loops are nested, each of which potentially visits each node once, then we say the algorithm is $O(n^2)$. If a divide-and-conquer approach is used, as in a *binary search* (where half of the list is eliminated at each step), then we would expect that only $\lfloor \log_2(n) + 1 \rfloor$ elements will actually be visited by the algorithm in the worst case, and hence we refer to it as an $O(\log n)$ operation. If an algorithm executes a subalgorithm $n$ times, and the subalgorithm is $O(\log n)$, then the resulting algorithm would be $O(n \log n)$.

To select an appropriate container class, we should look at the operations that we expect to be most common, then select the container whose performance characteristics for those operations are most favorable. The most common orders you'll encounter are listed here from fastest to slowest: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^k)$ for $k > 2$.

We should also take the memory layout and usage characteristics of our containers into account. For example, an array (e.g., `int a[5]` or `std::vector`) stores its elements *contiguously* in memory and requires *no overhead storage* for anything other than the elements themselves. (Note that

a *dynamic* array does require a small fixed overhead.) On the other hand, a linked list (e.g., `std::list`) wraps each element in a "link" data structure that contains a pointer to the next element and possibly also a pointer to the previous element, for a total of up to 16 bytes of overhead per element on a 64-bit machine. Also, the elements in a linked list need not be contiguous in memory and often aren't. A contiguous block of memory is usually much more cache-friendly than a set of disparate memory blocks. Hence, for high-speed algorithms, arrays are usually better than linked lists in terms of cache performance (unless the nodes of the linked list are themselves allocated from a small, contiguous block of memory). But a linked list is better for situations in which speed of inserting and removing elements is of prime importance.

### 6.3.4 Building Custom Container Classes

Many game engines provide their own custom implementations of the common container data structures. This practice is especially prevalent in console game engines and games targeted at mobile phone and PDA platforms. The reasons for building these classes yourself include:

- *Total control*. You control the data structure's memory requirements, the algorithms used, when and how memory is allocated, etc.
- *Opportunities for optimization*. You can optimize your data structures and algorithms to take advantage of hardware features specific to the console(s) you are targeting; or fine-tune them for a particular application within your engine.
- *Customizability*. You can provide custom algorithms not prevalent in the C++ standard library or third-party libraries like Boost (for example, searching for the *n* most relevant elements in a container, instead of just the single most relevant).
- *Elimination of external dependencies*. Since you built the software yourself, you are not beholden to any other company or team to maintain it. If problems arise, they can be debugged and fixed immediately rather than waiting until the next release of the library (which might not be until after you have shipped your game!)
- *Control over concurrent data structures*. When you write your own container classes, you have full control over the means by which they are protected against concurrent access on a multithreaded or multicore system. For example, on the PS4, Naughty Dog uses lightweight "spin lock" mutexes for the majority of our concurrent data structures, because they work well with our fiber-based job scheduling system. A third-party container library might not have given us this kind of flexibility.

We cannot cover all possible data structures here, but let's look at a few common ways in which game engine programmers tend to tackle containers.

#### 6.3.4.1 To Build or Not to Build

We will not discuss the details of how to implement all of these data types and algorithms here—a plethora of books and online resources are available for that purpose. However, we *will* concern ourselves with the question of where to obtain implementations of the types and algorithms that you need. As game engine designers, we have basically three choices:

1. Build the needed data structures manually.
2. Make use of the STL-style containers provided by the C++ standard library.
3. Rely on a third-party library such as Boost (http://www.boost.org).

The C++ standard library and third-party libraries like Boost are attractive options, because they provide a rich and powerful set of container classes covering pretty much every type of data structure imaginable. In addition, these packages provide a powerful suite of template-based *generic algorithms*—implementations of common algorithms, such as finding an element in a container, which can be applied to virtually any type of data object. However, these implementations may not be appropriate for some kinds of game engines. Let's take a moment to investigate some of the pros and cons of each approach.

##### *The C++ Standard Library*

The benefits of the C++ standard library's STL-style container classes include:

- They offer a rich set of features.
- Their implementations are robust and fully portable.

However, these container classes also have some drawbacks, including:

- The header files are cryptic and difficult to understand (although the documentation is quite good).
- General-purpose container classes are often slower than a data structure that has been crafted specifically to solve a particular problem.
- A generic container may consume more memory than a custom-designed data structure.
- The C++ standard library does a lot of dynamic memory allocation, and it's sometimes challenging to control its appetite for memory in a way that is suitable for high-performance, memory-limited console games.

- The templated allocator system provided by the standard C++ library isn't flexible enough to allow these containers to be used with certain kinds of memory allocators, such as stack-based allocators (see Section 6.2.1.1).

The *Medal of Honor: Pacific Assault* engine for the PC made heavy use of what was known at the time as the standard template library (STL). And while *MOHPA* did have its share of frame rate problems, the team was able to work around the performance problems caused by STL (primarily by carefully limiting and controlling its use). OGRE, the popular object-oriented rendering library that we use for some of the examples in this book, also makes heavy use of STL-style containers. However, at Naughty Dog we prohibit the use of STL containers in runtime game code (although we do permit their use in offline tools code). Your mileage may vary: Using the STL-style containers provided by the C++ standard library on a game engine project is certainly feasible, but it should be used with care.

#### *Boost*

The Boost project was started by members of the C++ Standards Committee Library Working Group, but it is now an open source project with many contributors from across the globe. The aim of the project is to produce libraries that extend and work together with the standard C++ library, for both commercial and non-commercial use. Many of the Boost libraries have already been included in the C++ standard library as of C++11, and more components are included in the Standards Committee's Library Technical Report (TR2), which is a step toward becoming part of a future C++ standard. Here is a brief summary of what Boost brings to the table:

- Boost provides a lot of useful facilities not available in the C++ standard library.
- In some cases, Boost provides alternatives or work-arounds for problems with the design or implementation of some classes in the C++ standard library.
- Boost does a great job of handling some very complex problems, like smart pointers. (Bear in mind that smart pointers are complex beasts, and they can be performance hogs. Handles are usually preferable; see Section 16.5 for details.)
- The Boost libraries' documentation is usually excellent. Not only does the documentation explain what each library does and how to use it, but

in most cases it also provides an excellent in-depth discussion of the design decisions, constraints and requirements that went into constructing the library. As such, reading the Boost documentation is a great way to learn about the principles of software design.

If you are already using the C++ standard library, then Boost can serve as an excellent extension and/or alternative to many of its features. However, be aware of the following caveats:

- Most of the core Boost classes are templates, so all that one needs in order to use them is the appropriate set of header files. However, some of the Boost libraries build into rather large .lib files and may not be feasible for use in very small-scale game projects.
- While the worldwide Boost community is an excellent support network, the Boost libraries come with no guarantees. If you encounter a bug, it will ultimately be your team's responsibility to work around it or fix it.
- The Boost libraries are distributed under the Boost Software License. Read the license information (http://www.boost.org/more/license_info.html) carefully to be sure it is right for your engine.

#### *Folly*

Folly is an open source library developed by Andrei Alexandrescu and the engineers at Facebook. Its goal is to extend the standard C++ library and the Boost library (rather than to compete with these libraries), with an emphasis on ease of use and the development of high-performance software. You can read about it by searching online for the article entitled "Folly: The Facebook Open Source Library" which is hosted at https://www.facebook.com/. The library itself is available on GitHub here: https://github.com/facebook/folly.

#### *Loki*

There is a rather esoteric branch of C++ programming known as *template metaprogramming*. The core idea is to use the compiler to do a lot of the work that would otherwise have to be done at runtime by exploiting the template feature of C++ and in effect "tricking" the compiler into doing things it wasn't originally designed to do. This can lead to some startlingly powerful and useful programming tools.

By far the most well-known and probably most powerful template metaprogramming library for C++ is Loki, a library designed and written by Andrei

Alexandrescu (whose home page is at http://www.erdani.org). The library can be obtained from SourceForge at http://loki-lib.sourceforge.net.

Loki is extremely powerful; it is a fascinating body of code to study and learn from. However, its two big weaknesses are of a practical nature: (a) its code can be daunting to read and use, much less truly understand, and (b) some of its components are dependent upon exploiting "side-effect" behaviors of the compiler that require careful customization in order to be made to work on new compilers. So Loki can be somewhat tough to use, and it is not as portable as some of its "less-extreme" counterparts. Loki is not for the faint of heart. That said, some of Loki's concepts such as *policy-based programming* can be applied to any C++ project, even if you don't use the Loki library per se. I highly recommend that all software engineers read Andrei's ground-breaking book, *Modern C++ Design* [3], from which the Loki library was born.

### 6.3.5 Dynamic Arrays and Chunky Allocation

Fixed size C-style arrays are used quite a lot in game programming, because they require no memory allocation, are contiguous and hence cache-friendly, and support many common operations such as appending data and searching very efficiently.

When the size of an array cannot be determined a priori, programmers tend to turn either to *linked lists* or *dynamic arrays*. If we wish to maintain the performance and memory characteristics of fixed-length arrays, then the dynamic array is often the data structure of choice.

The easiest way to implement a dynamic array is to allocate an $n$-element buffer initially and then *grow* the list only if an attempt is made to add more than $n$ elements to it. This gives us the favorable characteristics of a fixed size array but with no upper bound. Growing is implemented by allocating a new larger buffer, copying the data from the original buffer into the new buffer, and then freeing the original buffer. The size of the buffer is increased in some orderly manner, such as adding $n$ to it on each grow, or doubling it on each grow. Most of the implementations I've encountered never shrink the array, only grow it (with the notable exception of clearing the array to zero size, which might or might not free the buffer). Hence the size of the array becomes a sort of "high water mark." The `std::vector` class works in this manner.

Of course, if you can establish a high water mark for your data, then you're probably better off just allocating a single buffer of that size when the engine starts up. Growing a dynamic array can be incredibly costly due to reallocation and data copying costs. The impact of these things depends on the sizes of the

buffers involved. Growing can also lead to fragmentation when discarded buffers are freed. So, as with all data structures that allocate memory, caution must be exercised when working with dynamic arrays. Dynamic arrays are probably best used during development, when you are as yet unsure of the buffer sizes you'll require. They can always be converted into fixed size arrays once suitable memory budgets have been established.

### 6.3.6 Dictionaries and Hash Tables

A dictionary is a table of key-value pairs. A value in the dictionary can be looked up quickly, given its key. The keys and values can be of any data type. This kind of data structure is usually implemented either as a binary search tree or as a hash table.

In a binary tree implementation, the key-value pairs are stored in the nodes of the binary tree, and the tree is maintained in key-sorted order. Looking up a value by key involves performing an $O(\log n)$ binary search.

In a hash table implementation, the values are stored in a fixed size table, where each slot in the table represents one or more keys. To insert a key-value pair into a hash table, the key is first converted into integer form via a process known as *hashing* (if it is not already an integer). Then an *index* into the hash table is calculated by taking the hashed key *modulo* the size of the table. Finally, the key-value pair is stored in the slot corresponding to that index. Recall that the *modulo* operator (`%` in C/C++) finds the remainder of dividing the integer key by the table size. So if the hash table has five slots, then a key of 3 would be stored at index 3 (`3 % 5 == 3`), while a key of 6 would be stored at index 1 (`6 % 5 == 1`). Finding a key-value pair is an $O(1)$ operation in the absence of collisions.

#### 6.3.6.1 Collisions: Open and Closed Hash Tables

Sometimes two or more keys end up occupying the same slot in the hash table. This is known as a *collision*. There are two basic ways to *resolve* a collision, giving rise to two different kinds of hash tables:

- *Open*. In an open hash table (see Figure 6.8), collisions are resolved by simply storing more than one key-value pair at each index, usually in the form of a linked list. This approach is easy to implement and imposes no upper bound on the number of key-value pairs that can be stored. However, it does require memory to be allocated dynamically whenever a new key-value pair is added to the table.
- *Closed*. In a closed hash table (see Figure 6.9), collisions are resolved via a process of *probing* until a vacant slot is found. ("Probing" means ap-

plying a well-defined algorithm to search for a free slot.) This approach is a bit more difficult to implement, and it imposes an upper limit on the number of key-value pairs that can reside in the table (because each slot can hold only one key-value pair). But the main benefit of this kind of hash table is that it uses up a fixed amount of memory and requires no dynamic memory allocation. Therefore, it is often a good choice in a console engine.

Confusingly, closed hash tables are sometimes said to use *open addressing*, while open hash tables are said to use an addressing method known as *chaining*, so named due to the linked lists at each slot in the table.

#### 6.3.6.2 Hashing

Hashing is the process of turning a key of some arbitrary data type into an integer, which can be used modulo the table size as an index into the table. Mathematically, given a key $k$, we want to generate an integer hash value $h$ using the hash function $H$ and then find the index $i$ into the table as follows:

$$h = H(k),$$
$$i = h \bmod N,$$

where $N$ is the number of slots in the table, and the symbol mod represents the *modulo* operation, i.e., finding the remainder of the quotient $h/N$.

If the keys are unique integers, the hash function can be the identity function, $H(k) = k$. If the keys are unique 32-bit floating-point numbers, a hash function might simply reinterpret the bit pattern of the 32-bit float as if it were a 32-bit integer.

```
U32 hashFloat(float f)
{
```

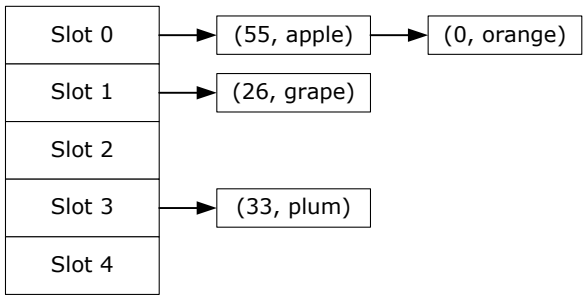


Figure 6.8. An open hash table.

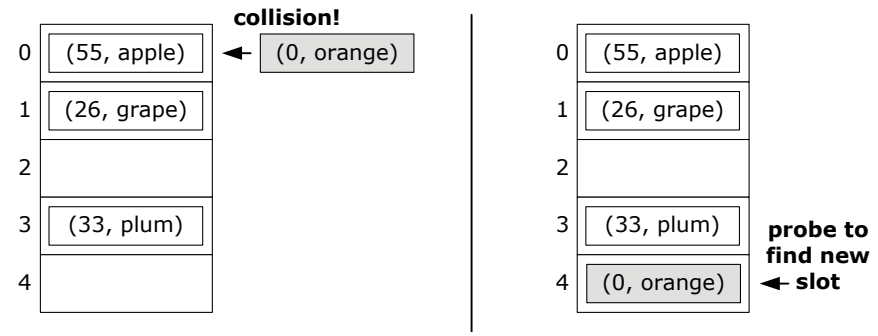


Figure 6.9. A closed hash table.

```
        union
        {
            float m_asFloat;
            U32   m_asU32;
        } u;

        u.m_asFloat = f;
        return u.m_asU32;
    }
```

If the key is a string, we can employ a *string hashing function*, which combines the ASCII or UTF codes of all the characters in the string into a single 32-bit integer value.

The *quality* of the hashing function $H(k)$ is crucial to the efficiency of the hash table. A "good" hashing function is one that distributes the set of all valid keys evenly across the table, thereby minimizing the likelihood of collisions. A hash function must also be reasonably quick to calculate, and *deterministic* in the sense that it must produce the exact same output every time it is called with an indentical input.

Strings are probably the most prevalent type of key you'll encounter, so it's particularly helpful to know a "good" string hashing function. Table 6.1 lists a number of well-known hashing algorithms, their throughput ratings (based on benchmark measurements and then converted into a rating of Low, Medium or High) and their score on the SMHasher test (https://github.com/aappleby/smhasher). Please note that the relative throughputs listed in the table are for rough comparison purposes only. Many factors contribute to the throughput of a hash function, including the hardware on which it is run and the properties of the input data. Cryptographic hashes are deliberately slow, as their focus is on producing a hash that is extremely unlikely to collide with the hashes of other input strings, and for which the task of determining

| Name | Throughput | Score | Cryptographic? |
|---|---|---|---|
| xxHash | High | 10 | No |
| MurmurHash 3a | High | 10 | No |
| SBox | Medium | 9 | No‡ |
| Lookup3 | Medium | 9 | No |
| CityHash64 | Medium | 10 | No |
| CRC32 | Low | 9 | No |
| MD5-32 | Low | 10 | Yes |
| SHA1-32 | Low | 10 | Yes |

Table 6.1. Comparison of well-known hashing algorithms in terms of their relative throughput and their scores on the SMHasher test. ‡Note that SBox is not itself a cryptographic hash, but it is one component of the symmetric key algorithms used in cryptography.

a string that would produce a given hash value is extremely computationally difficult.

For more information on hash functions, see the excellent article by Paul Hsieh available at http://www.azillionmonkeys.com/qed/hash.html.

#### 6.3.6.3 Implementing a Closed Hash Table

In a closed hash table, the key-value pairs are stored directly in the table, rather than in a linked list at each table entry. This approach allows the programmer to define a priori the exact amount of memory that will be used by the hash table. A problem arises when we encounter a *collision*—two keys that end up wanting to be stored in the same slot in the table. To address this, we use a process known as *probing*.

The simplest approach is *linear probing*. Imagine that our hashing function has yielded a table index of $i$, but that slot is already occupied; we simply try slots $(i+1)$, $(i+2)$ and so on until an empty slot is found (wrapping around to the start of the table when $i = N$). Another variation on linear probing is to alternate searching forwards and backwards, $(i+1)$, $(i-1)$, $(i+2)$, $(i-2)$ and so on, making sure to modulo the resulting indices into the valid range of the table.

Linear probing tends to cause key-value pairs to "clump up." To avoid these clusters, we can use an algorithm known as *quadratic probing*. We start at the occupied table index $i$ and use the sequence of probes $i_j = (i \pm j^2)$ for $j = 1, 2, 3, \ldots$. In other words, we try $(i+1^2)$, $(i-1^2)$, $(i+2^2)$, $(i-2^2)$ and so